

CSC-4890

Introduction to the Theory of Computation

Slides with thanks from Costas Busch

<http://www.csc.lsu.edu/~busch/courses/theorycomp/fall2015/>

General Info about Course

Books

- John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, Introduction to Automata Theory, Languages, and Computation
- Daniel I. A. Cohen, Introduction to Computer Theory
- Introduction to the Theory of Computation, Michael Sipser
- An Introduction to Formal Languages and Automata, Peter Linz

Other stuff

My own lecture notes!!!

Introduction to theory of Computations

- The theory of computation is a branch of theoretical computer science that deals with the study of algorithms and computational complexity. It aims to answer fundamental questions about
 - What can be computed
 - How efficiently it can be done.
 - What limitations exist in terms of computational power.

Language and Machines

- We need a language for communication with machines. But we do not require natural languages to communicate with the machines, as natural languages are very complex and machine interaction require very fewer complex languages compare to natural language.

Methods to Define Language

- In natural language we define the list of word in a dictionary because they are finite and predefined ,but we cannot list all sentence which can be formed using these words as they are infinite.
- So, we have mechanism called Grammer /rules using which we can decides which sentence

Mathematical Definition of Formal Language

- Symbols-symbols are the basic building blocks ,which can be any character /token.

Course Goals

Provide computation Models

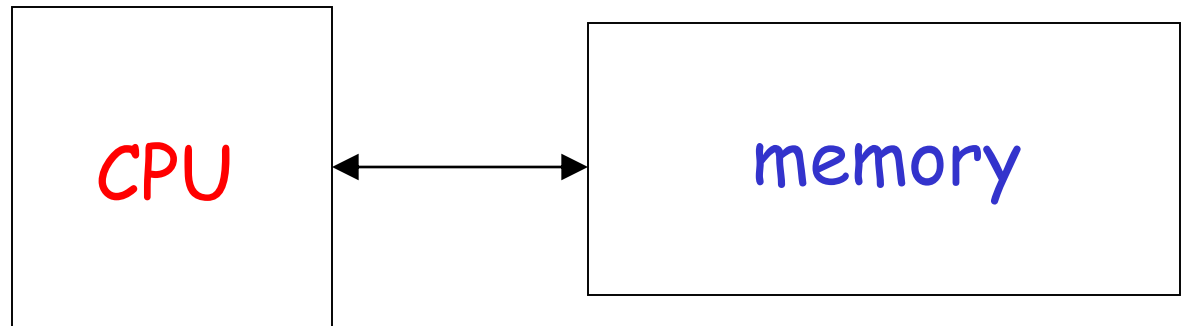
Analyze power of Models

Answer Intractability questions:

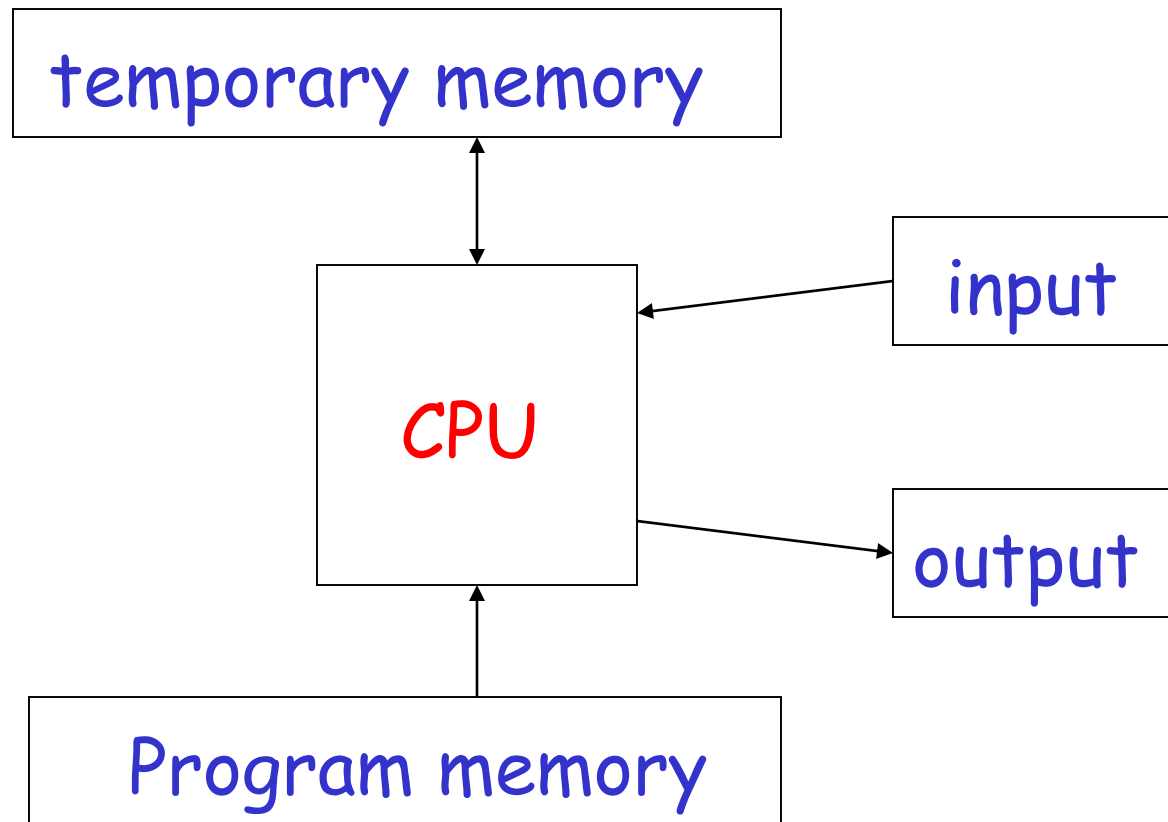
What computational problems
can each model solve?

[Not
Covered]

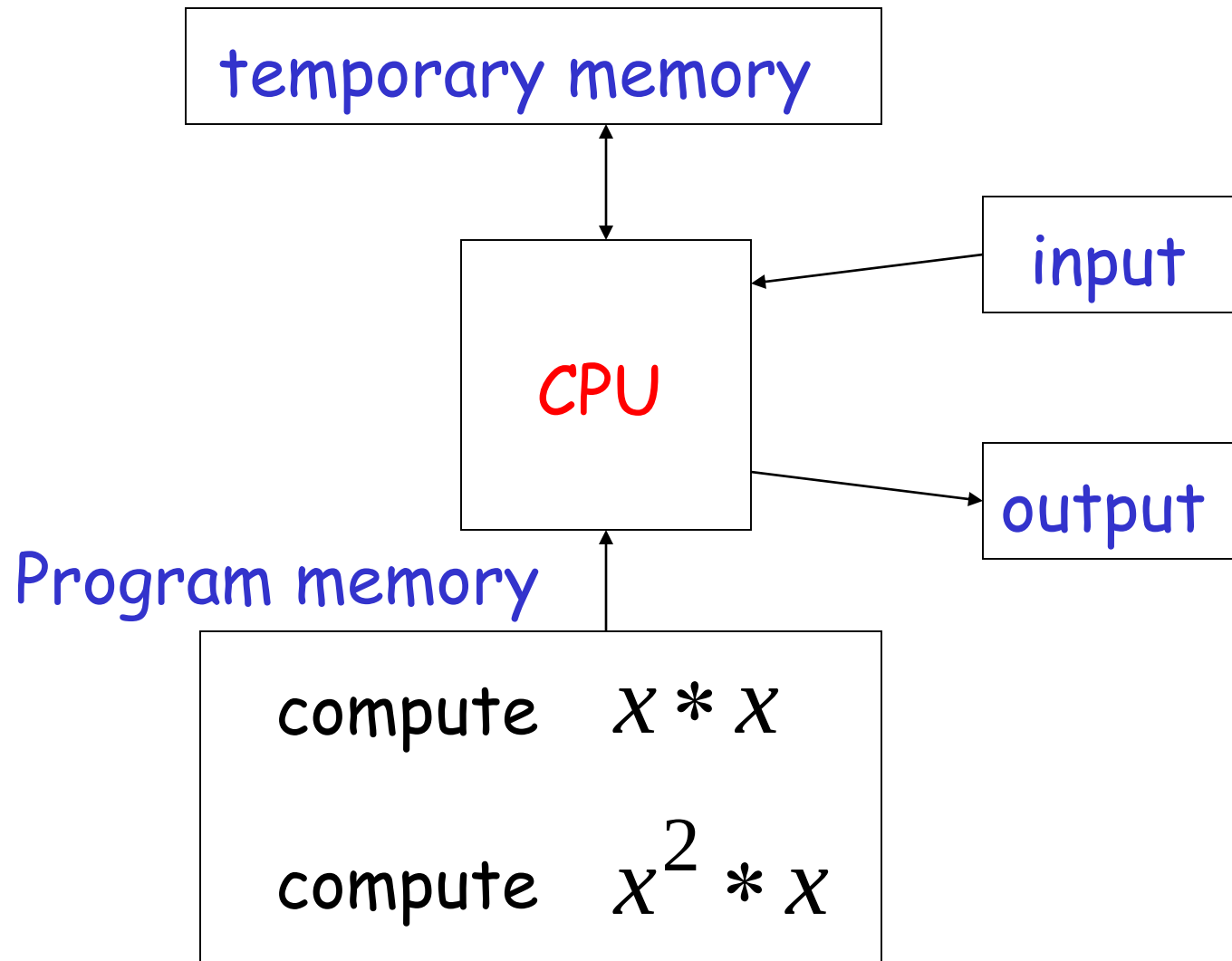
A widely accepted model of computation



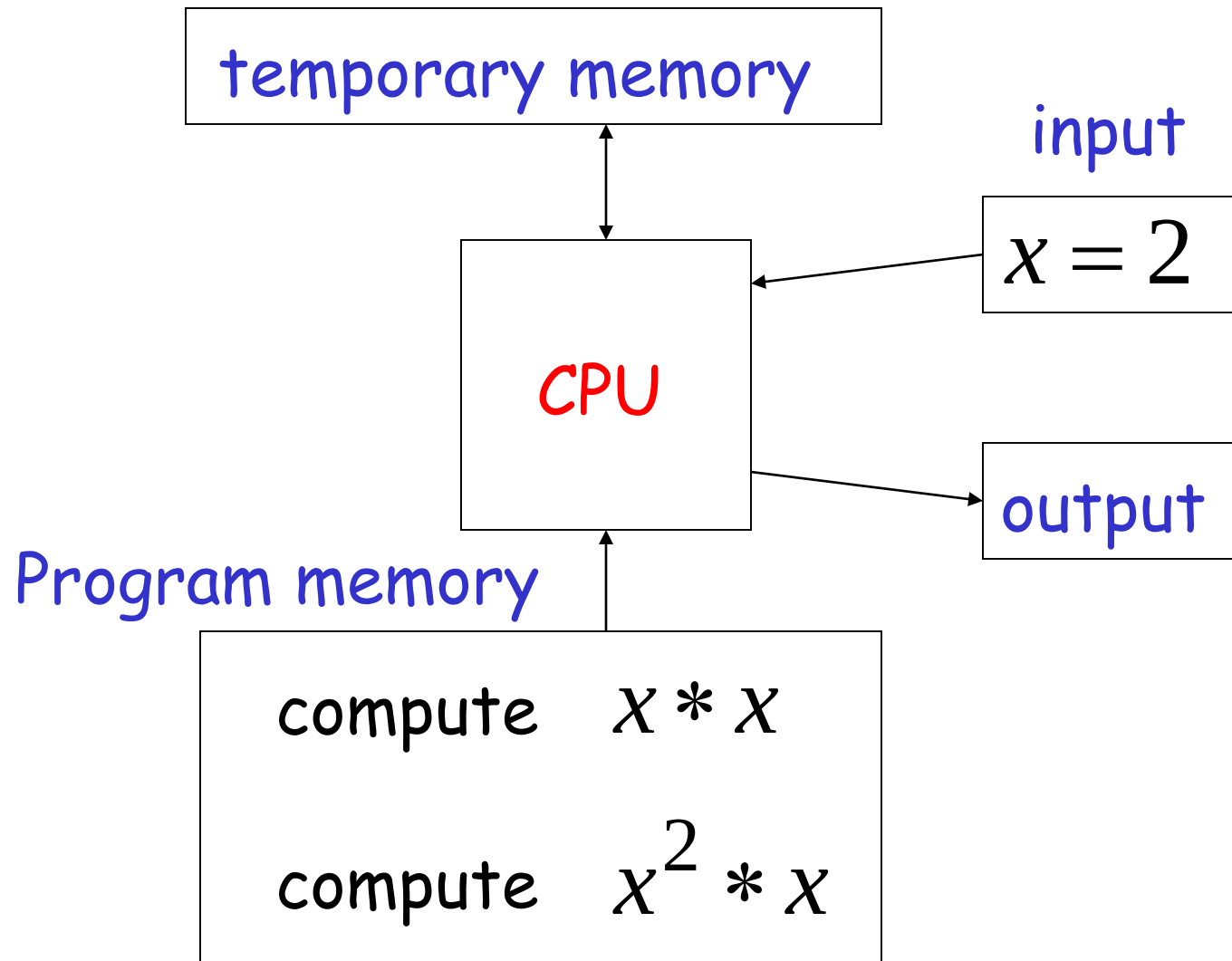
The different components of memory



Example: $f(x) = x^3$



$$f(x) = x^3$$



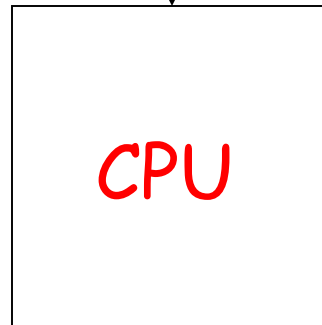
$$f(x) = x^3$$

temporary memory

$$z = 2 * 2 = 4$$
$$f(x) = z * 2 = 8$$

input

$$x = 2$$



output

Program memory

compute $x * x$

compute $x^2 * x$

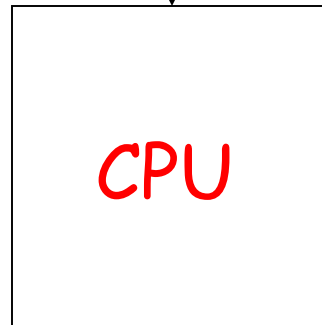
$$f(x) = x^3$$

temporary memory

$$z = 2 * 2 = 4$$
$$f(x) = z * 2 = 8$$

input

$$x = 2$$



$$f(x) = 8$$

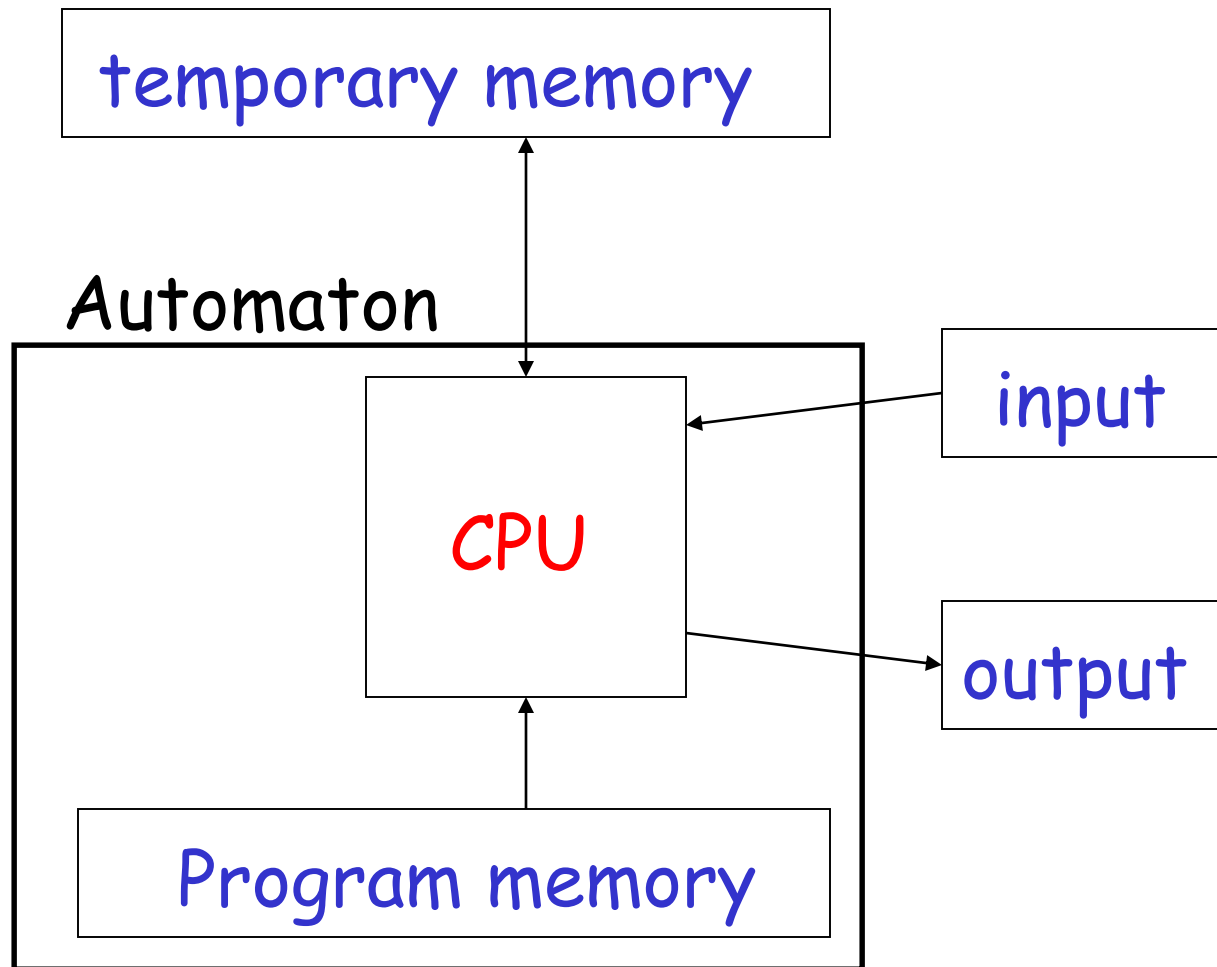
output

Program memory

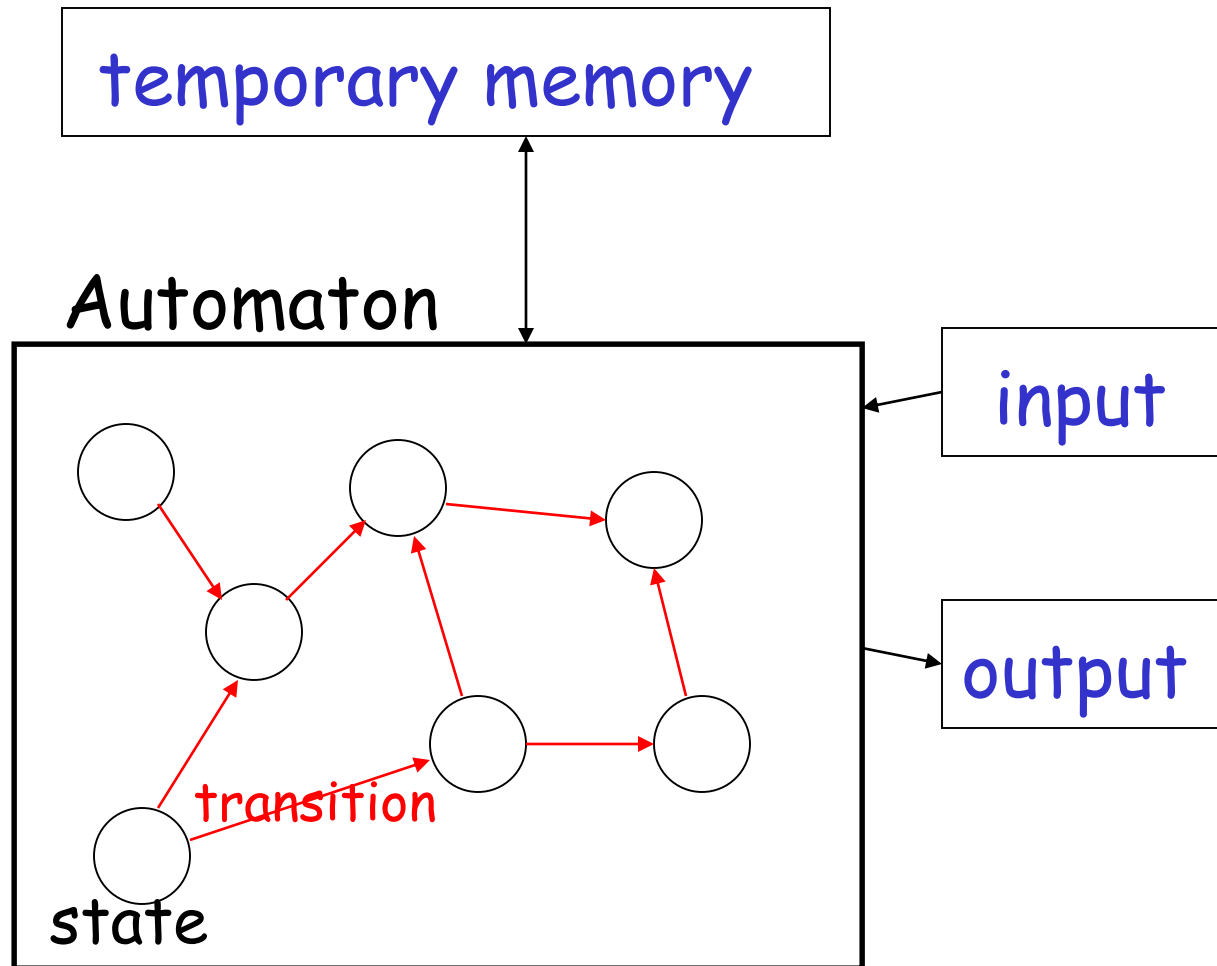
compute $x * x$

compute $x^2 * x$

Automaton



Automaton



$CPU + ProgramMem = States + Transitions$

Different Kinds of Automata

Automata are distinguished by the temporary memory

- **Finite Automata:** no temporary memory
- **Pushdown Automata:** stack
- **Turing Machines:** random access memory

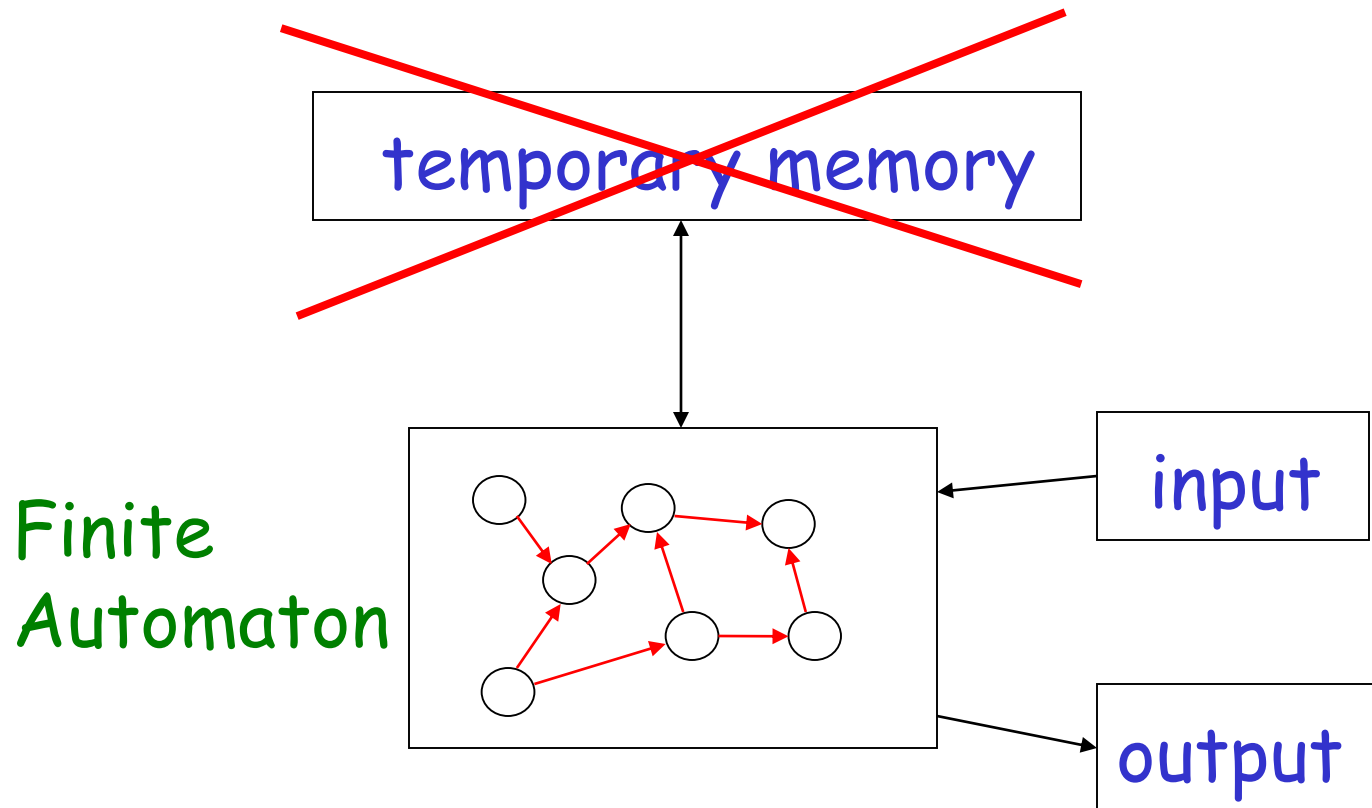
Memory affects computational power:

More flexible memory

results to

The solution of more computational
problems

Finite Automaton



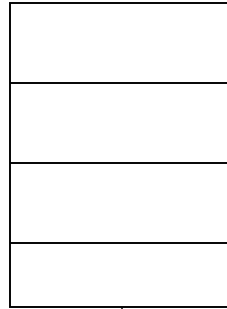
Example: Elevators, Vending Machines,
Lexical Analyzers
(small computing power)

Pushdown Automaton

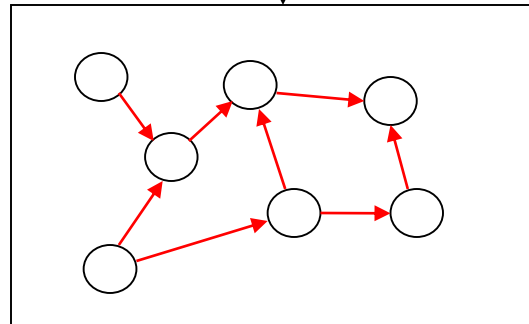
Temp.
memory

Stack

Push, Pop



Pushdown
Automaton



input

output

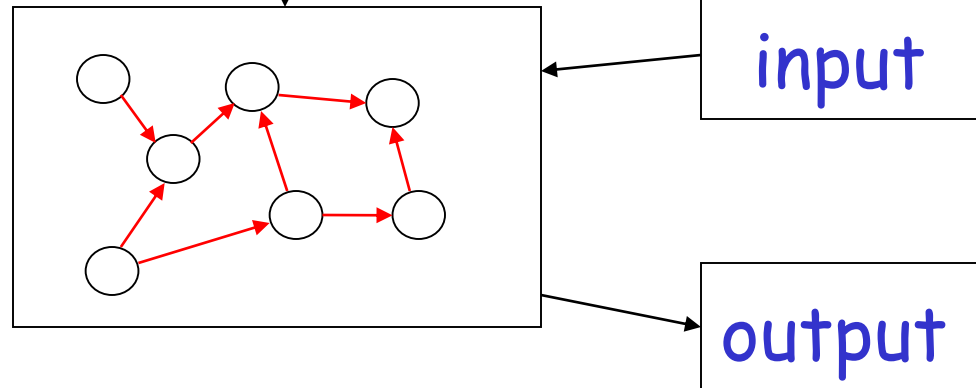
Example: Parsers for Programming Languages
(medium computing power)

Turing Machine

Temp.
memory

Random Access Memory

Turing
Machine



Examples: Any Algorithm

(highest known computing power)

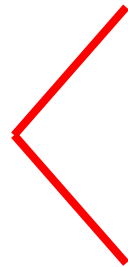
Power of Automata

Simple
problems

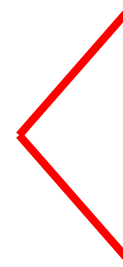
More complex
problems

Hardest
problems

Finite
Automata



Pushdown
Automata



Turing
Machine

Less power



More power

Solve more

computational problems

Turing Machine is the most powerful
known computational model

Question: can Turing Machines solve
all computational problems?

Answer: NO
(there are unsolvable problems)